COMPLEXITY THEORY

TRACTABLE AND INTRACTABLE PROBLEMS

Tractable problems are also known as easy problems. They can be solved in polynomial time

Eg: Binary Search ($O(\log n)$)

Bubble Sort ($O(n^2)$)

Intractable problems are also known as hard problems. They take super polynomial time to run to completion.

Eg: ~~TSP O(n!)~~ TSP ~~O(n!)~~ $O(2^n)$

N queens ~~O(n!)~~ $O(2^n)$

0/1 knapsack $O(2^n)$

Even though some problems look similar they do not necessarily fall in the category.

Eg: shortest path vs longest path

shortest path is a tractable problem whereas longest path is an intractable one

Eg: Euler's tour vs Hamiltonian Cycle

Euler's tour (visits every edge exactly once) is a tractable problem whereas Hamiltonian cycle (visits every vertices exactly once) is an intractable ~~one~~ one.

CLASS P AND NP

The class P consists of those problems that are solvable in polynomial time. More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k, where n is size of input to the problem.

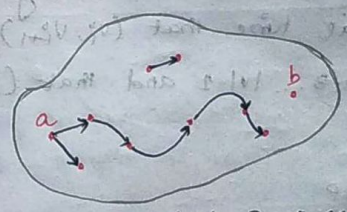
The class NP consists of those problems that are verifiable in polynomial time. If we were somehow given a certificate of a solution, then we could verify that the certificate is correct in polynomial time, polynomial in size of input to the problem.

Example: problem in P

PATH

Input: Graph G, nodes a and b

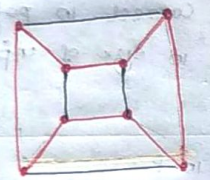
Question: Is there a path from a to b in G?



This problem is in P. To see if there is a path from node a to node b, one might determine all nodes reachable from

a by doing a BFS or dijkstras algorithm.

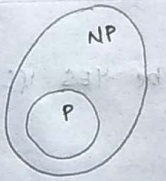
Example: Problem in NP



In hamiltonian cycle problem, given a directed graph $G(V,E)$, a certificate could be sequence $\langle v_1, v_2, v_3 \ldots v_{|V|} \rangle$ of of $|V|$ vertices. We could easily check in polynomial time that $(v_i, v_{i+1}) \in E$ for $i = 1, 2, 3 \ldots |V|-1$ and that $(v_{|V|}, v_1) \in E$ as well.

P vs NP

Any problem in P is also in NP, since if a problem is in P then we can solve it in polynomial time without even being supplied a certificate. So we can believe that $P \subseteq NP$.



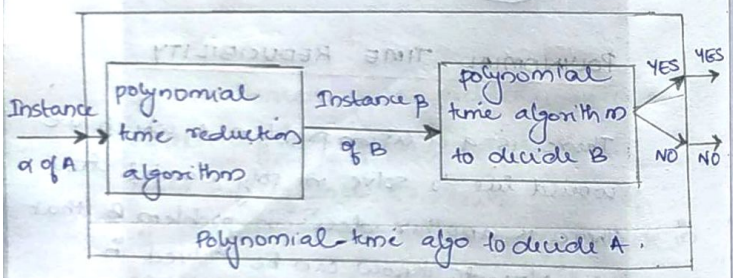
POLYNOMIAL TIME REDUCIBILITY

There is a decision problem A, which we would like to solve in polynomial time. There is another decision problem B that we already know can be solved in polynomial time. Then if all instances α of A can be transformed into some instance β of B within polynomial time then A too can be solved in polynomial time. This is known as polynomial time reduction.

The reduction should have the following properties

1) Transformation should be in polynomial time

2) Answer of A should be YES iff answer of B is also YES.



1. Given an instance α of problem A, we use a polynomial time reduction algorithm to transform it into an instance β of problem B

2. Run the polynomial time decision algo for B on instance β

3. Use the answer for α as the answer for β.

Each step independently take polynomial time So all the three step together take only polynomial time.

Note ⇒ Polynomial Time verification Algorithm:

It deals with the verification of the proof or outputs or certificate given to a particular problem.

A polynomial time verification algorithm is a 2-argument algorithm $A(x, y)$ where x is the input string and y is the binary certificate. The algorithm verifies y with respect to x in polynomial time.

Used in NP problems.

NP- Hard.

A language L is NP-HARD if $L' \leq_p L$ for every $L' \in NP$.

~~That is if all problems~~

That is L is NP-hard if all problems of NP can be reduced in polynomial time to L.

NP hardness is a class of problems that are informally, "at least as hard as the hardest problems in NP".

Eg: Hamiltonian cycle.

Every problem in NP is reducible to hamiltonian cycle in polynomial time.

Ex, TSP is reducible to HC.

NP Complete

A problem L is NP complete if

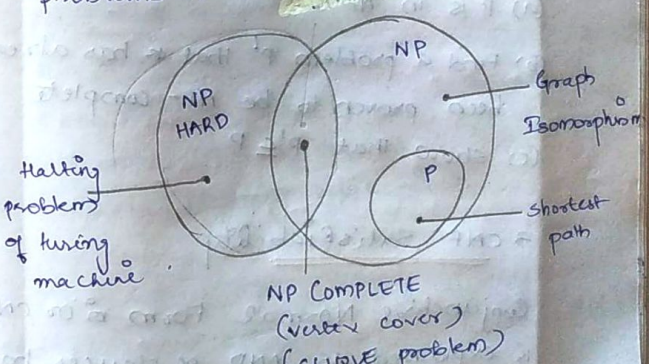
1. $L \in NP$ and

2. $L' \leq_p L$ for every $L' \in NP$.

That is a problem L is NP-complete if L is a problem in NP and all problems in NP can be reduced to problem L.

That is if L is NP and NP hard.

Class NP complete consists of the hardest problems in NP.



Proving NP - Completeness.

Method 1 (direct proof)

(a) P is in NP.

(b) All problems in NP-complete can be reduced to P.

Method 2

(a) P is in NP

(b) Find a problem P' that has already been proven to be NP-complete

(c) show that $P' \leq P$

3 CNF Satisfiability

Conjunctive Normal Form or CNF is explained as AND of clauses having one or more literals connected by OR.

eg: $(x_1 \vee x_2) \wedge (x_4 \vee \neg x_7)$

3 CNF has clauses with ~~exactly~~ atmost 3 literals ~~in each~~:

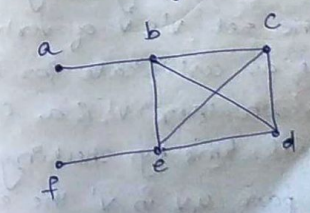
$(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_7)$

$\Rightarrow$ 3CNF is NP Complete.

The Clique Problem.

Clique of a graph $G=(V,E)$ is a complete sub graph of the graph G. That is, it is a subset $V' \subseteq V$ vertices such that for every pair of vertices in V', there is a corresponding edge in G.

Eg:



CLIQUE $= \{b, c, d, e\}$

In a clique decision problem we simply ask wheather a clique of size 'k' exist in the graph. So an algorithm will generate a subset of the graph with 'k' vertices and checks wheather it is a clique. It takes super polynomial time.

TO PROVE:

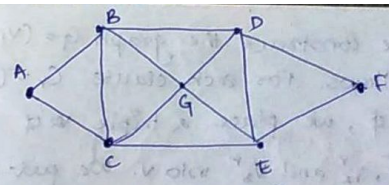
The clique problem is NP-complete

PROOF:

(a) To show that CLIQUE is NP complete we need to show CLIQUE $\in$ NP.

To check this for a given graph $G=(V,E)$ we use the set $V' \subseteq V$ of vertices in the clique as certificate for G. We can check wheather V' is a clique in polynomial time by checking wheather for each pair u,v in V' the edge (u,v) belongs to E.

Eg:



$V' = \{A, B, C, D, E\} \rightarrow$ CLIQUE

$V' = \{A, B, C, D, E, F, G\} \rightarrow$ NOT CLIQUE

(b) Next prove that 3 CNF $\leq_p$ CLIQUE which shows that clique problem is NP hard. We already know 3CNF is NP hard.

The reduction algorithm begins with an instance of 3CNF-SAT. Let $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ be a boolean formula in 3CNF with k clauses. For $r = 1, 2, \ldots k$ each clause C_r has exactly three distinct literals l_1^r, l_2^r and l_3^r. We shall construct a graph G such that ϕ is satisfiable if and only if G has clique of size k.

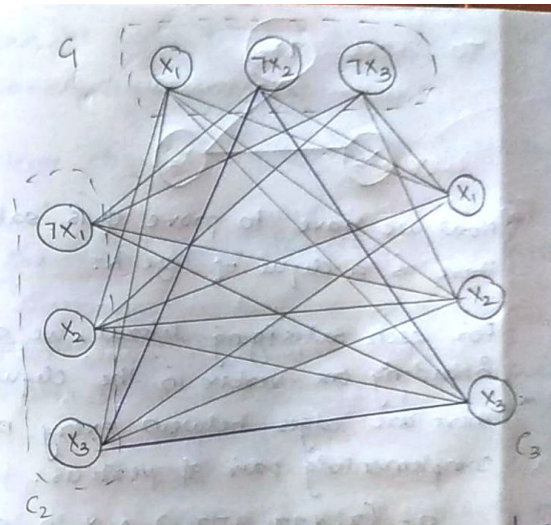
We construct the graph $G = (V, E)$ as follows. For each clause $C_r = (l_1^r \vee l_2^r \vee l_3^r)$ in ϕ, we place a triple of vertices v_1^r, v_2^r and v_3^r into V. We put an edge between two vertices v_i^r and v_j^s if both follows

(i) v_i^r and v_j^s are in different triples that is $r \neq s$ and

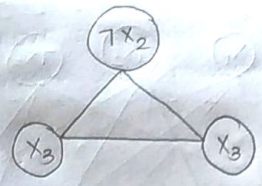
(ii) their corresponding literals are consistent, that is, l_i^r is not the negation of l_j^s.

We can easily build this graph from ϕ in polynomial time. As an example of this construction if we have

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$



The satisfiable assignment for ϕ can be obtained when $x_2 = 0$, $x_3 = 1$ and $x_1 = 0/1$. So from the first triple $\neg x_2$ is included in clique because it satisfy the clause 1. From clause 2 & 3 the literal x_3 is selected for inclusion in the clique.



Now we have to prove ϕ is satisfied, there is a clique of size k.

For each satisfying literal, we are including one vertex in the clique and there are edges between every non-complementing pair of vertices.

$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge x_2 = 0$ 1

$(\neg x_1 \vee x_2 \vee x_3) \wedge x_3 = 1$ 1

$(x_1 \vee x_2 \vee x_3)$ $x_3 = 1$ 1

$\phi = 1 \wedge 1 \wedge 1$

$= 1$

Now we have to prove if G has a clique, ϕ has a satisfying assignment. For that every vertex in G is mapped to the corresponding literal in the clause and can be given a 1 so that ϕ has a satisfying assignment.

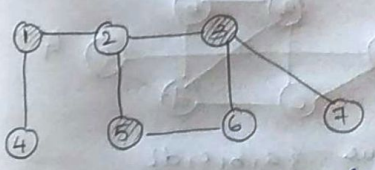
Hence clique problem is NP Complete.

The Vertex Cover Problem

For an undirected graph $G = (V, E)$, the vertex cover V' is the subset of V such that for any edge u, v, $u \in V'$ or $v \in V'$ or both.

It takes super polynomial time to prove compute vertex cover of the graph.

Eg:



vertex cover = $\{1, 5, 3\}$

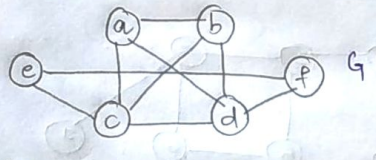
TO PROVE :

Vertex cover problem is np complete

PROOF :

(a) The first thing to prove is that vertex cover is NP. Given a set of vertices it can be verified in polynomial time wheather there is an ~~edge~~ end point of any edge (u,v) in the list of vertices.

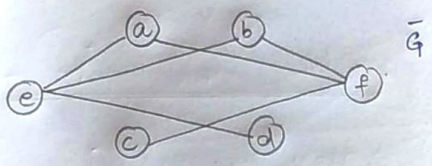
(b) To prove it is NP Complete, we have to reduce CLIQVE problem to vertex cover in polynomial time.



clique = {a, b, c, d}

Graph G is given. Compute the clique of graph G as V'.

Now construct $\bar{G}$ which contains all edges not in G. ~~and~~ And compute vertex cover of $\bar{G}$.



vertex cover = {e, f}

ie (V - V')

It can be seen that vertex cover of $\bar{G}$, $\bar{G}$ = V - V'. That is the clique problem can be reduced to vertex cover in polynomial time.

TO PROVE:

Travelling salesman problem is np-complete

PROOF:

(a) ~~The first~~ To show TSP is np complete first, we need to show TSP $\in$ NP. Given a set of vertices, we can check wheather this sequence contains each vertex exactly once, and sum up the cost and check wheather it is minimum. This can be verified in polynomial time.

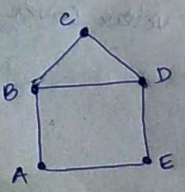
Hence, TSP $\in$ NP.

(b) To show TSP is np-hard we show that hamiltonian cycle can be reduced to TSP in polynomial time. ie

HAM_CYCLE $\leq_p$ TSP.

Consider a graph $G = (V, E)$ which has hamiltonian cycle.

$G =$

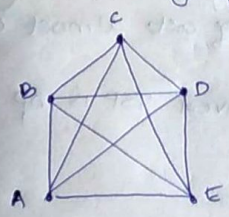


HAM_CYCLE = $\{A, B, C, D, E\}$.

~~Now construct a graph G' (V, E') which is the comp~~

Now construct a graph $G' = (V, E')$ which is the complete graph of G.

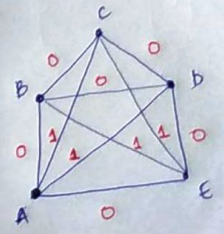
$G' =$



Define cost function such that

$$c(i,j) = \begin{cases} 0 & \text{if } (i,j) \in E \\ 1 & \text{if } (i,j) \in E' \end{cases}$$

$\therefore$



Now apply TSP in above graph is A-B-C-D-E with cost = 0.

∴

We now know that graph G has a hamiltonian cycle if and only if graph G' has a tour of cost atmost 0.

Hence we proved TSP is np complete.